

PHẦN 8.6 - GENERIC METHOD TRONG C#

Giải thích toàn tập: từ vấn đề thực tế đến ứng dụng trong .NET

Tài liệu học C# - Collections & Generics

Mục tiêu: hiểu Generic Method bằng bản chất, không học thuộc ký hiệu <T>.

1. Vấn đề Generic giải quyết

Giả sử bạn cần một method lấy phần tử đầu tiên trong danh sách.

Với int:

```
public static int GetFirstNumber(List<int> numbers)
{
    return numbers[0];
}
```

Với string:

```
public static string GetFirstName(List<string> names)
{
    return names[0];
}
```

Với Student:

```
public static Student GetFirstStudent(List<Student> students)
{
    return students[0];
}
```

Ba method trên có cùng một logic: nhận danh sách, lấy items[0], rồi trả về phần tử. Chỉ có kiểu dữ liệu thay đổi: int, string hoặc Student.

Đây là lúc Generic có ích. Ta viết một lần:

```
public static T GetFirst<T>(List<T> items)
{
    return items[0];
}
```

Sau đó có thể dùng cho List<int>, List<string>, List<Student>, List<Product>, List<Order>...

Generic cho phép viết một logic một lần và sử dụng lại với nhiều kiểu dữ liệu, nhưng vẫn giữ an toàn kiểu.

2. T thực sự là gì?

Trong public static T GetFirst<T>(List<T> items), T là type parameter - tham số đại diện cho kiểu dữ liệu.

Nó giống parameter bình thường, nhưng thay vì nhận một giá trị, nó đại diện cho một kiểu.

```
public static void Print(string name)
```

Parameter name có thể nhận các giá trị như "Hùng", "An", "Bình".

```
public static void Print<T>(T value)
```

Còn T có thể được thay bằng int, string, Student, Product... khi method được gọi.

3. Cách hình dung việc thay thế T

```
public static T GetFirst<T>(List<T> items)
{
    return items[0];
}

List<int> numbers = new List<int> { 10, 20, 30 };
int number = GetFirst(numbers);
```

C# nhìn thấy List<int> nên suy luận T = int. Về mặt tư duy, bạn có thể hình dung method lúc này giống như:

```
public static int GetFirst(List<int> items)
{
    return items[0];
}
```

Với List<string>, T = string. Với List<Student>, T = Student. Đây chỉ là cách hình dung để học; compiler không cần viết lại source code của bạn theo cách này.

4. Cú pháp Generic Method

```
accessModifier returnType MethodName<T>(parameters)
{
}
```

Ví dụ:

```
public static void Display<T>(T value)
{
    Console.WriteLine(value);
}
```

Thành phần	Ý nghĩa
public	Phạm vi truy cập
static	Gọi bằng tên class
void	Không trả về giá trị
Display	Tên method
<T>	Khai báo type parameter T
T value	Parameter value có kiểu T

Điểm dễ quên nhất là <T> phải nằm sau tên method. Nếu không khai báo <T>, compiler không biết T là gì.

5. T không bắt buộc phải tên là T

```
public static void Display<TValue>(TValue value)
{
    Console.WriteLine(value);
}
```

Quy ước đặt tên thường gặp:

Tên	Ý nghĩa thường dùng
T	Kiểu chung
TItem	Kiểu phần tử
TKey	Kiểu khóa
TValue	Kiểu giá trị
TResult	Kiểu kết quả
TEntity	Entity

Bạn đã gặp Dictionary<TKey, TValue>. Đây cũng là một ví dụ về nhiều type parameter.

6. Type inference - C# tự suy luận T

```
public static void Display<T>(T value)
{
    Console.WriteLine(value);
}

Display<int>(10);
Display<string>("Hello");
```

Thông thường có thể viết gọn:

```
Display(10);           // T = int
Display("Hello");      // T = string
Display(5.5m);         // T = decimal
```

C# nhìn vào argument để suy ra T. Cơ chế đó được gọi là type inference.

7. Generic method nhận một parameter kiểu T

```
public static void DisplayValue<T>(T value)
{
    Console.WriteLine(value);
}

DisplayValue(10);
DisplayValue("C#");
DisplayValue(9.5);
```

Một method có thể được gọi với nhiều kiểu khác nhau mà không cần viết nhiều overload trùng logic.

8. Hiện thị kiểu thật của T bằng typeof(T)

```
public static void DisplayValue<T>(T value)
{
    Console.WriteLine(
        $"Type: {typeof(T).Name}, Value: {value}");
}

DisplayValue(10);
DisplayValue("Hello");
DisplayValue(1_000_000m);
```

Kết quả có thể là:

```
Type: Int32, Value: 10
Type: String, Value: Hello
Type: Decimal, Value: 1000000
```

9. Generic method nhận nhiều parameter cùng kiểu

```
public static void DisplayPair<T>(T first, T second)
{
    Console.WriteLine(first);
    Console.WriteLine(second);
}

DisplayPair(10, 20);           // T = int
DisplayPair("C#", ".NET");    // T = string
```

Nếu gọi DisplayPair(10, "Hello"), C# có thể không suy luận được một T duy nhất vì hai argument có kiểu khác nhau.

10. Generic method trả về T

```
public static T GetFirst<T>(List<T> items)
{
    return items[0];
}
```

Nếu đầu vào là List<int> thì kết quả là int. Nếu là List<string> thì kết quả là string. Nếu là List<Student> thì kết quả là Student.

Quy tắc cần nhớ: đưa vào danh sách kiểu nào thì method trả ra một phần tử đúng kiểu đó.

11. GetFirst<T> có validation

```
public static T GetFirst<T>(List<T> items)
{
    if (items == null)
    {
```

```

        throw new ArgumentNullException(
            nameof(items),
            "The list cannot be null.");
    }

    if (items.Count == 0)
    {
        throw new InvalidOperationException(
            "The list is empty.");
    }

    return items[0];
}

```

Cần kiểm tra cả `items == null` và `items.Count == 0` trước khi truy cập `items[0]`.

12. GetLast<T>

```

public static T GetLast<T>(List<T> items)
{
    if (items == null)
    {
        throw new ArgumentNullException(nameof(items));
    }

    if (items.Count == 0)
    {
        throw new InvalidOperationException(
            "The list is empty.");
    }

    return items[items.Count - 1];
}

List<int> numbers = new() { 10, 20, 30 };
int last = GetLast(numbers); // 30

```

13. Generic method với array

Generic không chỉ dùng với `List<T>`.

```

public static T GetLast<T>(T[] items)
{
    if (items == null)
    {
        throw new ArgumentNullException(nameof(items));
    }

    if (items.Length == 0)
    {
        throw new InvalidOperationException(
            "Array is empty.");
    }

    return items[items.Length - 1];
}

int[] numbers = { 1, 2, 3 };
int result = GetLast(numbers);

string[] names = { "Hùng", "An", "Bình" };
string lastName = GetLast(names);

```

14. Swap<T> - ví dụ thấy rõ công năng Generic

Nếu không có Generic, bạn có thể phải viết `SwapInt()`, `SwapString()`, `SwapStudent()` dù logic đổi chỗ hoàn toàn giống nhau.

```

public static void Swap<T>(ref T first, ref T second)
{
    T temp = first;
    first = second;
    second = temp;
}

int a = 10;
int b = 20;
Swap(ref a, ref b);
// a = 20, b = 10

string first = "Hùng";
string second = "An";
Swap(ref first, ref second);
// first = "An", second = "Hùng"

```

15. Vì sao Swap cần ref?

Nếu method nhận T first, T second theo cách bình thường, việc gán lại hai biến chỉ thay đổi bản sao local bên trong method. ref cho phép method tác động trực tiếp lên biến được truyền vào.

```
Swap(ref a, ref b);
```

ref là cơ chế truyền tham chiếu của parameter. Generic chỉ giúp Swap hoạt động với nhiều kiểu dữ liệu.

16. Một method có thể có nhiều type parameter

```

public static void DisplayKeyValue<TKey, TValue>(
    TKey key,
    TValue value)
{
    Console.WriteLine(
        $"Key: {key}, Value: {value}");
}

DisplayKeyValue("SV01", "Hùng");
// TKey = string, TValue = string

DisplayKeyValue("SP01", 10_000_000m);
// TKey = string, TValue = decimal

DisplayKeyValue(1, student);
// TKey = int, TValue = Student

```

Đây là cùng tư duy mà Dictionary<TKey, TValue> sử dụng.

17. Generic method không bắt buộc trả về T

```

public static bool AreEqual<T>(T first, T second)
{
    return EqualityComparer<T>
        .Default
        .Equals(first, second);
}

bool result1 = AreEqual(10, 10); // true
bool result2 = AreEqual("C#", ".NET"); // false

```

Parameter có kiểu T, nhưng return type là bool. Generic Method vẫn có thể trả về một kiểu cố định khác T.

18. Vì sao không dùng first == second?

Compiler không biết T sẽ là kiểu gì. Không phải mọi T đều hỗ trợ toán tử == theo cách Generic Method có thể sử dụng trực tiếp.

```

EqualityComparer<T>.Default.Equals(
    first,
    second);

```

Đây là một cách so sánh tổng quát. `EqualityComparer<T>.Default` lấy bộ so sánh mặc định phù hợp với kiểu `T`, còn `Equals(first, second)` trả về `true` hoặc `false`.

19. Generic Method và Overload

Overload:

```
public static void Display(int value) { }
public static void Display(string value) { }
public static void Display(double value) { }
```

Generic:

```
public static void Display<T>(T value)
{
}
```

Nên dùng	Khi nào
Generic	Logic giống nhau, chỉ khác kiểu dữ liệu
Overload	Mỗi kiểu cần cách xử lý thực sự khác nhau

```
public static void Format(int value)
{
    Console.WriteLine($"Integer: {value}");
}

public static void Format(DateTime value)
{
    Console.WriteLine(value.ToString("dd/MM/yyyy"));
}
```

20. Generic khác object như thế nào?

```
public static void Display(object value)
{
}
```

object có thể nhận nhiều kiểu nhưng làm mất thông tin kiểu cụ thể. Khi lấy kết quả, bạn thường phải cast lại.

```
object result = GetValue();
Student student = (Student)result;
```

Với Generic:

```
Student student = GetFirst(students);
```

Compiler nhìn thấy `List<Student>`, suy luận `T = Student` và biết kết quả cũng là `Student`. Không cần ép kiểu thủ công.

21. Ba lợi ích lớn của Generic

1. Tái sử dụng code: một method thay cho nhiều method chỉ khác kiểu.
2. Type safety: compiler kiểm tra kiểu ngay khi biên dịch.
3. Không cần ép kiểu thủ công như khi dùng object.

22. Generic bạn đã sử dụng từ trước

```
List<int>           // T = int
List<Student>      // T = Student
Queue<Order>       // T = Order
Stack<string>      // T = string
HashSet<int>       // T = int
Dictionary<string, Student>
// TKey = string, TValue = Student
```

Khác nhau chỉ là: `List<T>` là Generic Class, còn `GetFirst<T>()` là Generic Method.

23. Hạn chế quan trọng của T

```
public static T Add<T>(T first, T second)
{
    return first + second;
}
```

Đoạn trên thường không hợp lệ vì T có thể là int, decimal, string, Student hoặc Order. Compiler không thể mặc định rằng mọi T đều hỗ trợ +, -, *, /, > hoặc <.

24. Không thể tùy ý gọi method trên T

```
public static void Print<T>(T value)
{
    value.DisplayInfo();
}
```

Compiler không biết mọi T có DisplayInfo() hay không. Nếu T = int thì 10.DisplayInfo() không tồn tại. Đây là lý do sau này cần Generic Constraint.

25. Không thể tùy ý tạo new T()

```
public static T Create<T>()
{
    return new T();
}
```

Compiler không chắc T có constructor không tham số. Phải yêu cầu:

```
public static T Create<T>()
    where T : new()
{
    return new T();
}
```

26. Generic Constraint dùng để làm gì?

Constraint có nghĩa: tôi chấp nhận Generic, nhưng T phải đáp ứng điều kiện được chỉ định.

Constraint	Ý nghĩa
where T : class	T phải là reference type
where T : struct	T phải là value type
where T : new()	T phải có constructor không tham số
where T : Employee	T phải là Employee hoặc class con
where T : IPrintable	T phải implement IPrintable

Phần 8.8 sẽ học Generic Constraints chi tiết. Hiện tại chỉ cần hiểu mục đích của chúng.

27. Generic Method có thể nằm trong class thường

```
public static class GenericHelper
{
    public static void Display<T>(T value)
    {
        Console.WriteLine(value);
    }
}

GenericHelper.Display(10);
```

28. Generic Method cũng có thể là instance method

```
public class Printer
{

```



```

    public void Display<T>(T value)
    {
        Console.WriteLine(value);
    }
}

```

```

Printer printer = new Printer();
printer.Display(10);
printer.Display("Hello");

```

Nếu method không cần dữ liệu trạng thái của object, static thường phù hợp hơn.

29. Generic Class và Generic Method khác nhau

Loại	Ví dụ	Phạm vi dùng T
Generic Method	GetFirst<T>()	Chỉ method sử dụng T
Generic Class	Storage<T>	Cả class sử dụng T

```

public class Storage<T>
{
    private T? _value;
}

```

30. Generic Method trong .NET thực tế

JSON

```

Student? student =
    JsonSerializer.Deserialize<Student>(json);

Product? product =
    JsonSerializer.Deserialize<Product>(json);

```

Một method Deserialize dùng cho nhiều kiểu object.

Entity Framework Core

```

dbContext.Set<Student>();
dbContext.Set<Product>();

```

Dependency Injection

```

services.AddScoped<
    IProductService,
    ProductService>();

```

LINQ

```

Where<T>()
First<T>()
ToList<T>()

```

Async

```

Task<Student>
Task<Product>
Task<List<Student>>

```

31. Cách nhận biết lúc nào nên dùng Generic

Khi code, hãy tự hỏi: "Mình có đang viết lại cùng một logic chỉ vì kiểu dữ liệu khác nhau không?"

Ví dụ	Kết luận
GetFirstInt, GetFirstString, GetFirstStudent	Generic rất phù hợp
SwapInt, SwapString, SwapStudent	Generic rất phù hợp

32. Ví dụ hoàn chỉnh GenericHelper

```
public static class GenericHelper
{
    public static void DisplayValue<T>(T value)
    {
        Console.WriteLine(
            $"Type: {typeof(T).Name}, Value: {value}");
    }

    public static T GetFirstItem<T>(List<T> items)
    {
        if (items == null)
        {
            throw new ArgumentNullException(
                nameof(items),
                "The list cannot be null.");
        }

        if (items.Count == 0)
        {
            throw new InvalidOperationException(
                "The list cannot be empty.");
        }

        return items[0];
    }

    public static T GetLastItem<T>(List<T> items)
    {
        if (items == null)
        {
            throw new ArgumentNullException(
                nameof(items),
                "The list cannot be null.");
        }

        if (items.Count == 0)
        {
            throw new InvalidOperationException(
                "The list cannot be empty.");
        }

        return items[items.Count - 1];
    }

    public static void SwapValues<T>(
        ref T first,
        ref T second)
    {
        T temp = first;
        first = second;
        second = temp;
    }

    public static bool AreValuesEqual<T>(
        T first,
        T second)
    {
        return EqualityComparer<T>
            .Default
            .Equals(first, second);
    }
}
```

```

public static void DisplayKeyValue<TKey, TValue>(
    TKey key,
    TValue value)
{
    Console.WriteLine(
        $"Key: {key}, Value: {value}");
}

```

33. Phân tích GetFirstItem<T>

```
public static T GetFirstItem<T>(List<T> items)
```

Vị trí	Ý nghĩa
<T>	Khai báo method sử dụng type parameter T
List<T>	Parameter là danh sách chứa kiểu T
T trước tên method	Method trả về kiểu T

Đọc bằng tiếng Việt: nhận một danh sách chứa kiểu gì thì trả về một phần tử đúng kiểu đó.

34. Phân tích SwapValues<T>

```

public static void SwapValues<T>(
    ref T first,
    ref T second)

```

Đọc bằng tiếng Việt: nhận hai biến cùng kiểu bất kỳ và đổi giá trị của chúng. T có thể là int, string, Student hoặc một kiểu khác.

35. Phân tích <TKey, TValue>

```

public static void DisplayKeyValue<TKey, TValue>(
    TKey key,
    TValue value)

```

Method cần hai kiểu độc lập. Kiểu của key không bắt buộc giống kiểu của value.

```

DisplayKeyValue("SP01", 500_000m);
// TKey = string, TValue = decimal

```

36. Lỗi thường gặp số 1 - quên khai báo <T>

Sai:

```

public static T GetValue(T value)
{
    return value;
}

```

Đúng:

```

public static T GetValue<T>(T value)
{
    return value;
}

```

37. Lỗi số 2 - khai báo <T> nhưng không dùng

```

public static void Display<T>(string value)
{
    Console.WriteLine(value);
}

```

T không đóng vai trò gì. Hoặc bỏ Generic, hoặc đổi parameter thành T value.

38. Lỗi số 3 - giả định T có chức năng nào đó

```
public static void Print<T>(T item)
{
    item.DisplayInfo();
}
```

Không thể bảo đảm mọi T đều có DisplayInfo(). Muốn yêu cầu điều đó cần constraint với class cha hoặc interface.

39. Lỗi số 4 - không validation collection

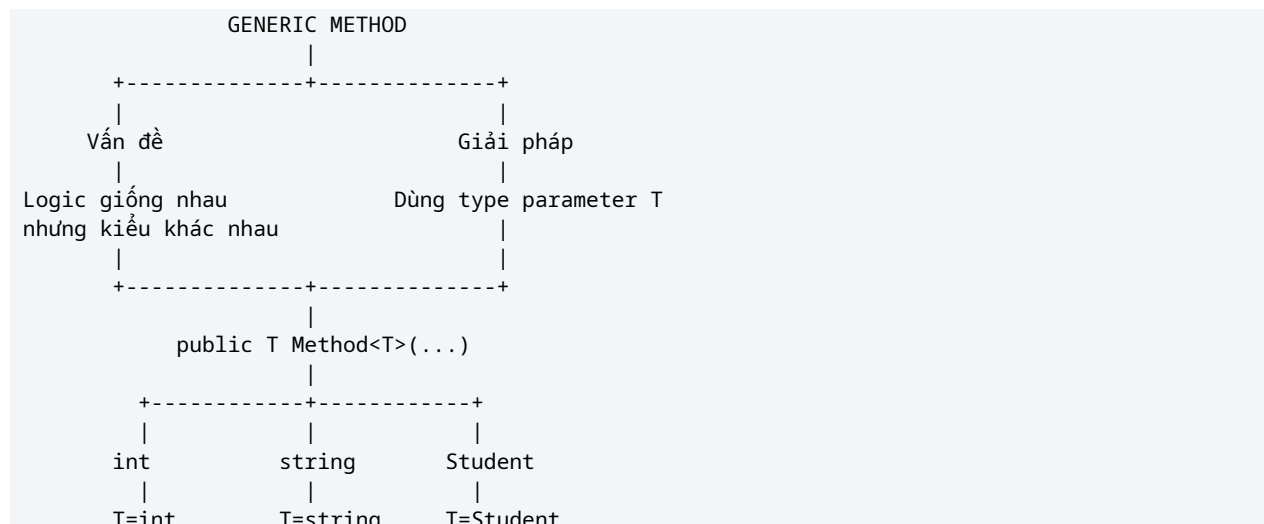
Không nên truy cập items[0] ngay lập tức. Cần kiểm tra items == null và items.Count == 0 trước.

40. Lỗi số 5 - dùng Generic khi không cần thiết

```
public static void SayHello<T>()
{
    Console.WriteLine("Hello");
}
```

T không được dùng nên Generic không có ý nghĩa. Chỉ cần SayHello() bình thường.

41. Sơ đồ tư duy toàn phần



42. Mối quan hệ với những phần đã học

```
List<T>
Dictionary<TKey, TValue>
HashSet<T>
Queue<T>
Stack<T>
```

Tất cả đều dựa trên Generic. Bạn đang chuyển từ sử dụng Generic do .NET viết sẵn sang tự viết Generic Method của mình.

43. Những điều thực sự cần thuộc

4. Generic Method giải quyết trường hợp logic giống nhau nhưng kiểu dữ liệu khác nhau.
5. Cú pháp cơ bản: public static void Display<T>(T value).
6. Method có thể trả về T: public static T GetFirst<T>(List<T> items).
7. C# có thể tự suy luận T từ argument.
8. Một method có thể có nhiều type parameter như <TKey, TValue>.
9. Không được mặc định rằng T có toán tử +, property, method hoặc constructor cụ thể.
10. Muốn yêu cầu khả năng của T thì dùng Generic Constraints.

44. Câu kiểm tra nhanh trước khi làm bài

11. Vì sao cần Generic? - Để tái sử dụng cùng một logic cho nhiều kiểu dữ liệu nhưng vẫn đảm bảo type safety.
12. Trong `GetFirst<T>()`, T là gì? - Là type parameter đại diện cho kiểu thật khi method được gọi.
13. Nếu truyền `List<Student>` vào `GetFirst<T>()` thì T là Student.
14. Generic Method chỉ dùng T trong method; Generic Class dùng T cho cả class.
15. Generic tốt hơn object ở chỗ giữ kiểu thật và thường không cần cast.
16. Không thể tùy ý viết first + second vì compiler không biết mọi T có hỗ trợ + hay không.

Kết luận: Generic Method là method có type parameter như T, cho phép cùng một logic hoạt động với nhiều kiểu dữ liệu mà vẫn giữ an toàn kiểu.

Cách nhận biết: cùng logic + chỉ khác kiểu dữ liệu -> hãy nghĩ đến Generic.